Guide to the Structure and Usage of the Spectral Computation Code

Guide to the Code Structure and Workflow

This note summarizes the purpose of the files, the main execution path, and the most important implementation details of the current code.

1. Overall purpose of the code

The code computes sampled spectral bands of periodic approximants associated with substitution-generated patterns. Conceptually, the workflow is

$$\text{substitution rule} \to \text{iterated patch} \to \text{finite Hamiltonian}$$

$$\to \text{Bloch phase sampling} \to \text{band edges} \to \text{saving and plotting.}$$

More concretely, the main execution chain is

$$\texttt{rule_defn_read} \to \texttt{iterate_sub} \to \texttt{save_bands} \to \texttt{sample_band_edges}$$

$$\to \texttt{op_mat_no_phase} \to \texttt{eigvalsh}.$$

For plotting, the workflow is

$$\texttt{read_bands} \to \texttt{print_several_bands}.$$

2. Practical interpretation

The code computes the sampled spectral bands of a periodic approximant as follows:

(a): Generate a substitution patch from a seed `start_tile` of size $a \times b$, producing after n iterations a patch of size $(a \cdot d^n) \times (b \cdot d^n)$.

(b): Flatten the patch row by row into a one-dimensional index set.

(c): Build the finite Schrödinger matrix with on-site potentials and nearest-neighbour hopping.

(d): Add Bloch wrap-around terms at the horizontal and vertical boundaries.

(e): Sample the ordered eigenvalues over all rational phase pairs of the form

$$\left(\pi\frac{a}{q}, \pi\frac{a'}{q'}\right) \qquad \text{with} \qquad q, q' \leq \texttt{res}.$$

(f): For each eigenvalue branch, record the sampled minimum and maximum.

(g): Save and plot the resulting band intervals.

3. File structure

The code is organized as follows.

(a): `main.py`

This is the driver script. It reads the substitution rule, generates iterated patches, computes the sampled spectral bands, and plots them.

(b): `Sub_Rule.py`

This file implements the substitution dynamics:

`rule_defn_read`, `sub_on_patch`, `iterate_sub`.

(c): `Aux_matrix.py`

This file contains the core matrix and eigenvalue routines:

`sub_diag`, `nested_phase_points`, `op_mat_no_phase`, `sample_band_edges`.

(d): `Aux_bands.py`

This file contains the band storage and plotting routines, together with

`get_ordinal_suffix`.

Hence the current dependency structure is

`main.py` → `Aux_bands.py` → `Aux_matrix.py`, `main.py` → `Sub_Rule.py`.

4. What happens when `main.py` is run?

When `main.py` is executed, the workflow is as follows:

(a): Initialise all parameters: `val`, `SUB_DEG`, `RULE_NAME`, `max_itera`, `res`, `init_itera`, `fin_itera`, and the starting seed `start_tile`.

Note: `len(val)` must match the number of distinct tile symbols in the substitution rule, given by `len(tile_rule)`.

(b): Read the substitution rule from an `.npy` file:

```
tile_rule = rule_defn_read(RULE_NAME).
```

(c): For each `it` $= 0, 1, \ldots,$ `max_itera`, generate the substituted patch and compute and save the sampled bands:

```
patch = iterate_sub(it, SUB_DEG, start_tile, tile_rule),
save_bands(it, res, patch, val).
```

(d): Plot the saved bands from `init_itera` to `fin_itera`:

```
print_several_bands(...).
```

So the main call chain is

`main` → `save_bands` → `sample_band_edges` → `op_mat_no_phase` → `eigvalsh`,

and for plotting,

`main` → `print_several_bands` → `read_bands`.

4.1 The function `rule_defn_read`

The function

`rule_defn_read(RULE_NAME)`

loads a substitution rule from a `.npy` file and returns it as a three-dimensional integer array `tile_rule`.

Input. `RULE_NAME` is a string specifying the name of the substitution rule, e.g. `"Table_Rule"` or `"Chair_Rule"`. The function looks for a file named

`RULE_NAME.npy`

in the current working directory.

Output. The function returns the array

$$\texttt{tile_rule}[s, \ell, k], \qquad s \in \{0, \ldots, S-1\}, \quad \ell, k \in \{0, \ldots, d-1\},$$

where S is the number of tile symbols and $d = \texttt{SUB_DEG}$ is the substitution degree. For each symbol s, the block $\texttt{tile_rule}[s, \cdot, \cdot]$ is the $d \times d$ substitution image of s.

5. The substitution and index numbering in patches

5.1 Storing patches and flattening procedure

A patch is stored as a two-dimensional array

$$\texttt{patch}[i][j],$$

where

(a): i is the row index,

(b): j is the column index,

(c): $i = 0$ is the top row,

(d): $j = 0$ is the left column.

Thus the patch is read row by row from top left to bottom right.
To build the Hamiltonian matrix, the two-dimensional coordinates are flattened via

$$(i, j) \mapsto k = i \cdot \texttt{wid} + j,$$

where `wid` is the number of columns. Hence the site numbering is row-major:

$$0, 1, 2, \ldots, \texttt{wid} - 1$$

in the first row,

$$\texttt{wid}, \ldots, 2\texttt{wid} - 1$$

in the second row, and so on. For a 4×4 patch the numbering is

$$\begin{matrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \\ 12 & 13 & 14 & 15 \end{matrix}$$

5.2 The function `sub_on_patch`

The function

```
sub_on_patch(sub_deg, patch, tile_rule)
```

applies one step of the block substitution rule to a patch.

Input.

(a): `patch` is a two-dimensional integer array of size

$$\texttt{wid}[0] \times \texttt{wid}[1],$$

where $\texttt{wid}[0] = \texttt{len(patch)}$ is the number of rows and $\texttt{wid}[1] = \texttt{len(patch[0])}$ is the number of columns.

(b): $\texttt{sub_deg} = d$ is the substitution degree.

(c): `tile_rule` is a three-dimensional integer array of shape

$$\texttt{tile_rule}[s, \ell, k], \qquad s \in \{0, \ldots, S-1\}, \quad \ell, k \in \{0, \ldots, d-1\},$$

where S is the number of tile symbols. For each symbol s, the $d \times d$ block

$$\texttt{tile_rule}[s, \cdot, \cdot]$$

is the substitution image of s.

Output. The function returns a new array `arr` of size

$$(d \cdot \texttt{wid}[0]) \times (d \cdot \texttt{wid}[1]),$$

initialised to -1 and then filled by the substitution rule.

Construction. For each site (i, j) in the original patch, and for each offset (ℓ, k) with $0 \le \ell, k < d$, the output is defined by

$$\texttt{arr}[i \cdot d + \ell,\ j \cdot d + k] \ = \ \texttt{tile_rule}[\texttt{patch}[i, j],\ \ell,\ k].$$

That is, each tile symbol $\texttt{patch}[i, j] = s$ is replaced by the $d \times d$ block $\texttt{tile_rule}[s, \cdot, \cdot]$, placed at position $(i \cdot d, j \cdot d)$ in the output array.

Example. For $d = 2$ and a 2×2 patch

$$\texttt{patch} = \begin{pmatrix} s_{00} & s_{01} \\ s_{10} & s_{11} \end{pmatrix},$$

the output is the 4×4 array

$$\texttt{arr} = \begin{pmatrix} T[s_{00}] & T[s_{01}] \\ T[s_{10}] & T[s_{11}] \end{pmatrix},$$

where each $T[s] = \texttt{tile_rule}[s, \cdot, \cdot]$ is the 2×2 substitution block of symbol s, and the block matrix is to be read entry-wise, so that `arr` has size 4×4.

5.3 The function `iterate_sub`

The function

```
iterate_sub(iter_num, sub_deg, patch, tile_rule)
```

applies the substitution rule `iter_num` times by iterating `sub_on_patch`.

Input.

(a): `iter_num` $= n \geq 0$ is the number of substitution steps.

(b): `sub_deg`, `patch`, `tile_rule` are as described above.

Output.

(a): If $n = 0$, the original `patch` is returned unchanged.

(b): If $n \geq 1$, the function returns

$$\texttt{sub_on_patch}^n(\texttt{patch}),$$

i.e. the result of applying `sub_on_patch` exactly n times in succession.

Patch growth. If the initial patch has size $a \times b$, then after n iterations the patch has size $(a \cdot d^n) \times (b \cdot d^n)$. In the standard case $d = 2$, this gives $(a \cdot 2^n) \times (b \cdot 2^n)$.

6. The role of `Aux_matrix.py`

The file `Aux_matrix.py` serves as the central layer for matrix construction and eigenvalue computations. Its tasks are:

(a): assign potential values to the symbols in a patch,

(b): generate the sampled Bloch phase set,

(c): build the phase-free Hamiltonian matrix,

(d): compute the sampled band edges for all eigenvalue branches.

The current functions in this file are

`sub_diag`, `nested_phase_points`, `op_mat_no_phase`, `sample_band_edges`.

6.1 The function `sub_diag`

The function

```
sub_diag(index, val)
```

returns the potential value associated with the patch symbol `index`.
If

$$\texttt{patch}[i][j] = k,$$

then the diagonal entry of the Hamiltonian at this site is

$$\texttt{val}[k].$$

The function checks that the index is valid:

$$0 \leq \texttt{index} < \texttt{len(val)}.$$

If this fails, an `IndexError` is raised.
Thus `sub_diag` is the map

$$\text{symbol in the patch} \;\rightarrow\; \text{on-site potential.}$$

6.2 The function `nested_phase_points`

The function

```
nested_phase_points(res)
```

builds the phase set used in the Bloch sampling.
The phase grid is not dyadic. The function does not restrict to fractions of the form

$$\frac{a}{2^r}.$$

Instead, it takes all rational numbers

$$\frac{a}{q} \qquad \text{with} \qquad 0 \leq a \leq q, \quad 1 \leq q \leq \texttt{res},$$

and maps them to

$$\theta = \pi \frac{a}{q}.$$

So the phase set is

$$\Theta_{\texttt{res}} = \left\{ \pi \frac{a}{q} : \; 0 \leq a \leq q, \; 1 \leq q \leq \texttt{res} \right\},$$

with duplicates removed. This happens because:

(a): `Fraction(a,q)` automatically reduces fractions.

(b): The reduced fractions are stored in a Python `set`, which keeps only unique values.

Hence one may also write

$$\Theta_{\texttt{res}} = \left\{ \pi \frac{a}{q} \in [0, \pi] : \; \gcd(a, q) = 1, \; 1 \leq q \leq \texttt{res} \right\}.$$

For $\texttt{res} = 4$, the resulting rational values are

$$0, \; \frac{1}{4}, \; \frac{1}{3}, \; \frac{1}{2}, \; \frac{2}{3}, \; \frac{3}{4}, \; 1,$$

so the phase set is

$$0, \; \frac{\pi}{4}, \; \frac{\pi}{3}, \; \frac{\pi}{2}, \; \frac{2\pi}{3}, \; \frac{3\pi}{4}, \; \pi.$$

Thus $n_{\text{pts}} := |\Theta_{\texttt{res}}|$ is in general not equal to $\texttt{res} + 1$.

6.3 The function `op_mat_no_phase`

The function

$$\texttt{op_mat_no_phase(patch, val)}$$

builds the phase-free Hamiltonian matrix.
Let

$$\texttt{height} = \texttt{len(patch)}, \qquad \texttt{wid} = \texttt{len(patch[0])}, \qquad N = \texttt{height} \cdot \texttt{wid}.$$

Then the matrix has size

$$H_0 \in \mathbb{C}^{N \times N}.$$

For the site (i, j) with flattened index

$$k = i \cdot \texttt{wid} + j,$$

the function assigns:

(a): the diagonal term

$$(H_0)_{k,k} = \texttt{sub_diag}(\texttt{patch}[i][j], \texttt{val}),$$

(b): nearest-neighbour hopping terms of strength 1:

$$(H_0)_{k,k+1} = 1 \quad \text{if the right neighbour exists,}$$

$$(H_0)_{k,k-1} = 1 \quad \text{if the left neighbour exists,}$$

$$(H_0)_{k,k+\texttt{wid}} = 1 \quad \text{if the lower neighbour exists,}$$

$$(H_0)_{k,k-\texttt{wid}} = 1 \quad \text{if the upper neighbour exists.}$$

So `op_mat_no_phase` is the discrete Schrödinger operator on the rectangular grid with open boundary conditions.
For a 2×2 patch

$$\texttt{patch} = \begin{pmatrix} a & b \\ c & d \end{pmatrix},$$

with numbering

$$(0,0) \mapsto 0, \qquad (0,1) \mapsto 1, \qquad (1,0) \mapsto 2, \qquad (1,1) \mapsto 3,$$

the matrix is

$$H_0 = \begin{pmatrix} V_a & 1 & 1 & 0 \\ 1 & V_b & 0 & 1 \\ 1 & 0 & V_c & 1 \\ 0 & 1 & 1 & V_d \end{pmatrix},$$

where

$$V_a = \texttt{val}[a], \qquad V_b = \texttt{val}[b], \qquad V_c = \texttt{val}[c], \qquad V_d = \texttt{val}[d].$$

Note that the matrix is created with

```
dtype=complex.
```

This is necessary because Bloch boundary terms are complex. Without this, NumPy would discard imaginary parts when phase factors are added.

6.4 The function `sample_band_edges`

The function

```
sample_band_edges(res, patch, val)
```

is the central spectral routine. It takes as input:

(a): the resolution parameter `res`,

(b): the patch,

(c): the potential values `val`.

If the patch has size

$$\texttt{height} \times \texttt{wid},$$

then the Hamiltonian dimension is

$$\texttt{size} = \texttt{height} \cdot \texttt{wid}.$$

The substitution step enters only through the patch:

```
patch = iterate_sub(it, SUB_DEG, start_tile, tile_rule).
```

6.4.1 Internal structure

The routine proceeds as follows.

Step 1: build the phase set. It computes

$$\texttt{phases = nested_phase_points(res)}, \qquad n_{\text{pts}} = \texttt{len(phases)}.$$

Thus the sampled phase pairs are

$$(\theta_1, \theta_2) \in \Theta_{\texttt{res}} \times \Theta_{\texttt{res}}.$$

Step 2: build the phase-free base matrix. It computes once

```
mat1 = op_mat_no_phase(patch, val).
```

This is the open-boundary matrix H_0.

Step 3: precompute the exponentials. It computes

```
exp_pos = np.exp(1j * phases),    exp_neg = np.exp(-1j * phases).
```

So for each sampled phase θ, the values

$$e^{i\theta} \qquad \text{and} \qquad e^{-i\theta}$$

are available without repeated recomputation inside the inner loop.

Step 4: precompute boundary index pairs. The wrap-around geometry depends only on the size of the patch.

(a): Vertical wrap:

$$\text{top row} \leftrightarrow \text{bottom row},$$

with index pairs

$$(j,\ (\texttt{height}-1)\texttt{wid}+j), \qquad j=0,\dots,\texttt{wid}-1.$$

(b): Horizontal wrap:

$$\text{left column} \leftrightarrow \text{right column},$$

with index pairs

$$(i\texttt{wid},\ i\texttt{wid}+\texttt{wid}-1), \qquad i=0,\dots,\texttt{height}-1.$$

Step 5: loop over all phase pairs. For each pair

$$(\theta_1,\theta_2)=(\texttt{phases}[ki],\texttt{phases}[li]),$$

the code forms

$$\texttt{mat = mat1.copy()}.$$

Thus the phase-dependent updates do not alter the base matrix `mat1`.

Step 6: add Bloch boundary terms. The Bloch phase contributions are inserted directly inside `sample_band_edges`. There is no longer a separate function `op_mat_phase`. The horizontal wrap-around terms are

$$\texttt{mat[left,right]} \mathrel{+}= e^{-i\theta_1}, \qquad \texttt{mat[right,left]} \mathrel{+}= e^{i\theta_1}.$$

The vertical wrap-around terms are

$$\texttt{mat[top,bottom]} \mathrel{+}= e^{-i\theta_2}, \qquad \texttt{mat[bottom,top]} \mathrel{+}= e^{i\theta_2}.$$

Hence the full Bloch Hamiltonian is

$$H(\theta_1,\theta_2)=H_0+H_{\text{phase}}(\theta_1,\theta_2).$$

Step 7: diagonalize. The code computes

$$\texttt{vect = LA.eigvalsh(mat)}.$$

Since the matrix is Hermitian, `eigvalsh` returns the eigenvalues in increasing order:

$$\lambda_1(\theta_1,\theta_2)\le\lambda_2(\theta_1,\theta_2)\le\cdots\le\lambda_{\texttt{size}}(\theta_1,\theta_2).$$

Step 8: store the result. The code computes sampled minima and maxima for every branch:

$$\texttt{band_max}[k]=\max\lambda_{k+1}(\theta_1,\theta_2),$$
$$\texttt{band_min}[k]=\min\lambda_{k+1}(\theta_1,\theta_2),$$

where the extrema are taken over all

$$(\theta_1,\theta_2)\in\Theta^2_{\texttt{res}}.$$

Therefore, the k-th sampled band is

$$B_k^{(\text{sampled})}=[\texttt{band_min}[k],\ \texttt{band_max}[k]]\,.$$

7. The role of `Aux_bands.py`

The file `Aux_bands.py` contains the band storage, file naming, and plotting routines. The current functions in this file are

`save_bands`, `read_bands`, `print_several_bands`, `get_ordinal_suffix`.

7.1 The function `get_ordinal_suffix`

This function returns the English ordinal suffix associated with an integer. Examples include:

$$1 \mapsto \texttt{"st"}, \qquad 2 \mapsto \texttt{"nd"}, \qquad 3 \mapsto \texttt{"rd"}, \qquad 4 \mapsto \texttt{"th"}.$$

It also handles the exceptions

$$11, 12, 13 \mapsto \texttt{"th"}.$$

So

$$\text{suffix}(n) = \begin{cases} \texttt{th} & n \bmod 100 \in \{11, 12, 13\}, \\ \texttt{st} & n \bmod 10 = 1, \\ \texttt{nd} & n \bmod 10 = 2, \\ \texttt{rd} & n \bmod 10 = 3, \\ \texttt{th} & \text{otherwise.} \end{cases}$$

It is used only for file naming in `Aux_bands.py`, for example

`1st`, `2nd`, `3rd`, `4th`.

7.2 The function `save_bands`

The function

```
save_bands(it, res, patch, val)
```

computes the sampled band edges for a given patch and saves the result to disk as a `.npy` file.

Input.

(a): `it`: the current iteration number, used to construct the output filename via `get_ordinal_suffix`.

(b): `res`: the phase resolution parameter, passed to `sample_band_edges`.

(c): `patch`: the substituted patch at iteration `it`.

(d): `val`: the on-site potential value vector.

Output. The function calls

```
sample_band_edges(res, patch, val)
```

and saves the resulting array of band minima and maxima to a `.npy` file in the `Finished computations/` folder.

7.3 The function `read_bands`

The function

```
read_bands(it, val)
```

loads the saved band data for iteration `it` from disk.

Input.

(a): `it`: the iteration number identifying which file to load.

(b): `val`: the potential value vector, used to reconstruct the filename.

Output. The function returns the array saved by `save_bands`, containing the sampled band minima and maxima:

$$\texttt{mat}[0][k] = \texttt{band_min}[k], \qquad \texttt{mat}[1][k] = \texttt{band_max}[k].$$

7.4 The function `print_several_bands`

The function

```
print_several_bands(init_itera, fin_itera, val, init_size, ...)
```

plots the sampled spectral bands for a range of substitution iterations.

Input. The required parameters are:

(a): $\texttt{init_itera} \geq 0$: the first iteration to plot.

(b): $\texttt{fin_itera} \geq \texttt{init_itera}$: the last iteration to plot.

(c): `val`: the potential value vector, used to locate the saved data files.

(d): `init_size`: the number of eigenvalues at iteration 0. At iteration `it`, the number of bands is

$$\texttt{size} = \texttt{init_size} \times d^{2\,\texttt{it}}.$$

In the standard case $d = 2$ this gives $\texttt{init_size} \times 4^{\,\texttt{it}}$.

The optional parameters are:

(e): `vert_dist` (default 0.5): controls the visual vertical spacing between consecutive iterations by adjusting the figure height. It does *not* alter the y-data coordinates, which remain at $y = \texttt{init_itera}, \texttt{init_itera} + 1, \ldots, \texttt{fin_itera}$.

(f): `plot_tiny` (default `False`): if `True`, numerically degenerate bands (those with width below `tiny_tol`) are displayed as vertical markers instead of being omitted.

(g): `tiny_tol` (default 10^{-8}): threshold below which a band is considered degenerate:

$$\texttt{right} - \texttt{left} < \texttt{tiny_tol}.$$

(h): `tiny_height_factor` (default 0.15): height of the vertical degenerate-band marker, expressed as a fraction of one iteration step.

(i): `color` (default `'black'`): plot colour for all bands.

(j): `linewidth` (default 0.9): line thickness for both regular and degenerate bands.

(k): `fig_width` (default 8.0): figure width in inches.

(l): `save_path` (default `None`): if not `None`, the figure is saved to this file path.

Output. The function produces a matplotlib figure. The x-axis displays the spectral variable; the y-axis shows the iteration number `it`. Each band at iteration `it` is drawn as a horizontal line segment at height $y = \texttt{it}$, spanning the interval

$$[\,\texttt{band_min}[k],\ \texttt{band_max}[k]\,].$$

Internal structure. The routine proceeds as follows.

(a): The figure height is set to

$$\texttt{fig_height} = \max(2.0,\ \texttt{vert_dist} \times \texttt{n_levels} + 1.0),$$

where $\texttt{n_levels} = \texttt{fin_itera} - \texttt{init_itera} + 1$.

(b): For each $\texttt{it} \in \{\texttt{init_itera}, \ldots, \texttt{fin_itera}\}$, the saved band data is loaded via

```
mat = read_bands(it, val),
```

and for each eigenvalue branch $i = 0, \ldots, \texttt{size} - 1$, the band endpoints are recovered as

$$\texttt{left} = \min(\texttt{mat}[0][i],\ \texttt{mat}[1][i]), \qquad \texttt{right} = \max(\texttt{mat}[0][i],\ \texttt{mat}[1][i]).$$

(c): Bands with $\texttt{right} - \texttt{left} \geq \texttt{tiny_tol}$ are collected as horizontal line segments. Bands with $\texttt{right} - \texttt{left} < \texttt{tiny_tol}$ are collected as vertical markers (if `plot_tiny` is `True`).

(d): All segments are rendered efficiently using `matplotlib`'s `LineCollection`, which avoids repeated individual `plot` calls.

(e): The y-axis ticks are set to the natural iteration numbers $\texttt{init_itera}, \ldots, \texttt{fin_itera}$, with a margin of ± 0.5 added above and below.